

Python Interview Mastery

40 Concepts Every Developer Should Know

From mutability and the GIL to decorators and MRO — the concepts interviewers actually ask about, each explained with a real, runnable example.

`is vs ==``*args / **kwargs``decorators``GIL``MRO``context managers`

Most Interviews Test The Same Core Ideas

Python interviewers rarely ask about obscure trivia. Instead, the same 40 or so ideas resurface again and again — across junior screens, senior system-design rounds, and take-home reviews alike.

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)    # [1, 2, 3, 4]  -- why does changing b change a?
```

If you can explain **why** that snippet prints what it prints — reference semantics, mutability, and object identity — you already understand three of the concepts in this guide. That is exactly the kind of reasoning interviewers are listening for.

40 Concepts, Explained With Real Examples

Each of the next 40 cards covers one interview-favorite concept: a plain-English definition, a short runnable code example, and 2 key points worth saying out loud in an interview.

- ✓ Read the definition first — that is your one-sentence interview answer.
- ✓ Run the code example yourself; typing it out beats just reading it.
- ✓ The key points are the follow-up details that separate a good answer from a great one.

Four Categories Worth Knowing

1 Data Model & Memory

Concepts 01–10 — mutability, identity, copying, references, the GIL

2 Functions & Functional Style

Concepts 11–20 — *args, closures, decorators, generators, comprehensions

3 Object-Oriented Programming

Concepts 21–30 — self, dunder methods, inheritance, encapsulation

4 Errors, Scope & Environment

Concepts 31–40 — exceptions, context managers, scope, concurrency

01

Mutable vs Immutable Objects

Mutable objects can be changed after creation; immutable ones cannot. Lists, dicts, and sets are mutable — ints, strings, tuples, and frozensets are immutable.

```
x = [1, 2, 3]
x.append(4)          # same list, modified in place

y = (1, 2, 3)
y[0] = 9             # TypeError: tuples are immutable
```

- ✓ Immutable objects are safer to use as dict keys and inside sets.
- ✓ Reassigning a variable (`x = 5`) never mutates the original object — it just rebinds the name.

02

is vs == (Identity vs Equality)

== compares values; is compares identity — whether two names point to the exact same object in memory.

```
a = [1, 2, 3]
b = [1, 2, 3]
a == b      # True    (same values)
a is b      # False   (different objects)
a is a      # True
```

- ✓ Use is only for None, True, False — never for value comparison.
- ✓ Small ints (-5 to 256) and short strings may be cached, so is can give misleading results.

03

Shallow Copy vs Deep Copy

A shallow copy duplicates the outer object but keeps references to the same nested objects; a deep copy recursively duplicates everything.

```
import copy
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
deep = copy.deepcopy(original)

shallow[0].append(99)    # also changes original
deep[0].append(99)      # original stays untouched
```

- ✓ `list(x)` and `x[:]` also create shallow copies.
- ✓ Deep copies cost more memory and time — only use them when nested mutation is a real risk.

04

Pass by Object Reference

Python passes the reference to an object into a function — not the object itself, and not a raw memory address like C's pointers.

```
def modify(lst):  
    lst.append(4)          # mutates the original list  
  
def reassign(lst):  
    lst = [9, 9, 9]        # only rebinds the local name  
  
nums = [1, 2, 3]  
modify(nums)              # nums -> [1, 2, 3, 4]  
reassign(nums)            # nums is still [1, 2, 3, 4]
```

- ✓ Mutating an object inside a function affects the caller; reassigning the parameter does not.
- ✓ This model is often called “pass by object reference” or “pass by assignment.”

05

Python's Garbage Collection

CPython uses reference counting as its primary memory manager, backed by a cyclic garbage collector that catches what reference counting alone can't free.

```
import sys, gc

x = []
print(sys.getrefcount(x))    # current reference count
gc.collect()                 # force a collection cycle
```

- ✓ An object is freed the instant its reference count hits zero.
- ✓ Circular references (a points to b, b points to a) never reach zero alone — the cyclic collector handles those.

06

The GIL (Global Interpreter Lock)

The GIL is a mutex that allows only one thread to execute Python bytecode at a time, even on a multi-core machine.

```
import threading

def work():
    total = sum(i * i for i in range(10**6))

threads = [threading.Thread(target=work) for _ in range(4)]
[t.start() for t in threads]
[t.join() for t in threads]
# CPU-bound: threads don't truly run in parallel here
```

- ✓ CPU-bound work: use multiprocessing to bypass the GIL entirely.
- ✓ I/O-bound work (network, disk): threading still helps, since the GIL is released during I/O waits.

07

Tuple vs List

Both store ordered collections, but tuples are immutable and slightly faster; lists are mutable and more flexible.

```
point = (10, 20)           # tuple: fixed coordinates
scores = [90, 85, 77]      # list: can grow, shrink, sort
scores.append(100)
# point[0] = 5              # TypeError
```

- ✓ Tuples can be used as dictionary keys; lists cannot, since keys must be hashable.
- ✓ Use a tuple to signal “this data shouldn't change” — it documents intent.

08

Dictionary vs Set

A dict stores key-value pairs; a set stores only unique, unordered values. Both are hash-based, giving average $O(1)$ lookup.

```
prices = {"apple": 40, "banana": 20}    # dict
fruits = {"apple", "banana", "apple"}    # set ->
{'apple', 'banana'}

"apple" in prices    # True, checks keys
```

- ✓ Both require hashable elements/keys — no lists or dicts nested inside a set.
- ✓ Sets are ideal for membership tests and removing duplicates from a collection.

09

String Immutability & Interning

Strings can never be changed in place — every “modification” creates a new string object. CPython also interns some strings to save memory.

```
s = "hello"
s += " world"           # creates a brand-new string object

a = "python"
b = "python"
a is b                  # True — short identifier-like strings are
interned
```

- ✓ Repeated string concatenation in a loop is $O(n^2)$ — use `"".join(parts)` instead.
- ✓ Interning is a CPython implementation detail; don't rely on it for correctness.

10

Slicing

Slicing extracts a sub-sequence using [start:stop:step], without mutating the original sequence.

```
nums = [0, 1, 2, 3, 4, 5]
nums[1:4]      # [1, 2, 3]
nums[::-1]     # [5, 4, 3, 2, 1, 0]    reversed
nums[::2]      # [0, 2, 4]             every 2nd item
```

- ✓ stop is always exclusive — the same rule range() follows.
- ✓ A negative step reverses direction; a[:] makes a quick shallow copy.

11

*args and **kwargs

*args collects extra positional arguments into a tuple; **kwargs collects extra keyword arguments into a dict.

```
def greet(*args, **kwargs):  
    print(args)          # ('Alice', 'Bob')  
    print(kwargs)        # {'greeting': 'Hi'}  
  
greet("Alice", "Bob", greeting="Hi")
```

- ✓ The names args/kwargs are convention — the * and ** are what matter.
- ✓ Great for wrapper functions that forward arguments to another function unchanged.

12

Lambda Functions

A lambda is a small, anonymous, single-expression function — useful for short, throwaway logic.

```
square = lambda x: x * x
square(5)      # 25

data = [(1, "b"), (2, "a")]
sorted(data, key=lambda pair: pair[1])  # sort by 2nd item
```

- ✓ No statements allowed inside a lambda — only a single expression.
- ✓ Prefer a normal def for anything more complex than one line; it's easier to read and debug.

13

Closures

A closure is an inner function that remembers variables from its enclosing scope, even after the outer function has finished running.

```
def make_counter():  
    count = 0  
    def increment():  
        nonlocal count  
        count += 1  
        return count  
    return increment  
  
counter = make_counter()  
counter()    # 1  
counter()    # 2
```

- ✓ `nonlocal` lets the inner function modify (not just read) the enclosing variable.
- ✓ Closures are the mechanism behind function factories and many decorators.

14

Decorators

A decorator is a function that wraps another function to add behavior before or after it runs, without changing its source code.

```
def timer(func):
    def wrapper(*args, **kwargs):
        import time
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time()-start:.4f}s")
        return result
    return wrapper

@timer
def slow_add(a, b):
    return a + b
```

- ✓ @decorator above a function is exactly `func = decorator(func)`.
- ✓ Use `functools.wraps(func)` inside wrapper to preserve the original name and docstring.

15

Generators and yield

A generator function uses `yield` to produce a sequence of values lazily, one at a time, instead of building the whole list in memory.

```
def count_up_to(n):  
    i = 1  
    while i <= n:  
        yield i  
        i += 1  
  
for num in count_up_to(3):  
    print(num)    # 1, 2, 3
```

- ✓ Execution pauses at `yield` and resumes right there on the next call — state is kept automatically.
- ✓ Ideal for large or infinite sequences, since values are produced on demand.

16

Iterators vs Iterables

An iterable implements `__iter__` and can be looped over; an iterator implements `__next__` and tracks its own position, produced by calling `iter()` on an iterable.

```
nums = [1, 2, 3]      # iterable
it = iter(nums)        # iterator
next(it)    # 1
next(it)    # 2
```

- ✓ Every iterator is also an iterable (its `__iter__` returns itself), but not every iterable is an iterator.
- ✓ A for loop calls `iter()` once, then `next()` repeatedly until `StopIteration`.

17

List Comprehension vs Generator Expression

A list comprehension `[x for x in ...]` builds the entire list in memory at once; a generator expression `(x for x in ...)` yields items lazily.

```
squares_list = [x*x for x in range(1_000_000)]    # built now
squares_gen  = (x*x for x in range(1_000_000))    # built on
demand
```

- ✓ Use a generator expression when you only need to iterate once and want to save memory.
- ✓ Swap the brackets for parentheses — the syntax is otherwise identical.

18

map(), filter(), reduce()

map() applies a function to every item, filter() keeps items that satisfy a condition, and reduce() (from functools) folds a sequence into a single value.

```
from functools import reduce
nums = [1, 2, 3, 4]
list(map(lambda x: x*2, nums))           # [2, 4, 6, 8]
list(filter(lambda x: x % 2 == 0, nums)) # [2, 4]
reduce(lambda a, b: a + b, nums)         # 10
```

- ✓ map() and filter() return lazy iterators — wrap in list() to see all results.
- ✓ A comprehension often reads more clearly than map()/filter() for simple cases.

19

Recursion

A recursive function calls itself on a smaller version of the problem until it reaches a base case that stops the recursion.

```
def factorial(n):  
    if n <= 1:                # base case  
        return 1  
    return n * factorial(n - 1)  # recursive case  
  
factorial(5)    # 120
```

- ✓ Python's default recursion limit is 1000 frames deep (`sys.getrecursionlimit()`).
- ✓ Python does not optimize tail calls, so deep recursion can be slower and riskier than an equivalent loop.

20

The Walrus Operator (:=)

Introduced in Python 3.8, `:=` assigns a value to a variable as part of a larger expression, avoiding a separate assignment line.

```
data = [1, 2, 3, 4, 5]
if (n := len(data)) > 3:
    print(f"List has {n} items")

while (line := input("> ")) != "quit":
    print(line)
```

- ✓ Saves recomputing an expensive expression twice — once to check it, once to use it.
- ✓ Must be wrapped in parentheses in most contexts.

21

The self Keyword

self refers to the specific instance a method is being called on — it's how an object accesses and modifies its own attributes.

```
class Dog:
    def __init__(self, name):
        self.name = name
    def bark(self):
        return f"{self.name} says woof!"

Dog("Rex").bark()    # "Rex says woof!"
```

- ✓ self is a naming convention, not a keyword — Python passes the instance automatically as the first argument.
- ✓ Without self, a method has no way to read or set that instance's own attributes.

22

__init__ vs __new__

`__new__` actually creates and returns a new instance; `__init__` then initializes that already-created instance's attributes.

```
class Point:
    def __new__(cls, *args):
        print("Creating instance")
        return super().__new__(cls)
    def __init__(self, x, y):
        print("Initializing instance")
        self.x, self.y = x, y
```

```
Point(1, 2)
# Creating instance
# Initializing instance
```

- ✓ `__new__` runs first and is a staticmethod; `__init__` receives the object `__new__` returned.
- ✓ You rarely need to override `__new__` — mainly for immutable types or singletons.

23

Class vs Static vs Instance Methods

Instance methods take self and act on one object; class methods take cls and act on the class itself; static methods take neither and are just grouped inside the class.

```
class Pizza:
    count = 0
    def __init__(self):
        Pizza.count += 1
    @classmethod
    def total_made(cls):
        return cls.count
    @staticmethod
    def is_valid_size(size):
        return size in ("S", "M", "L")
```

- ✓ @classmethod is often used for alternative constructors, e.g. `Pizza.from_dict(data)`.
- ✓ @staticmethod is essentially a plain function namespaced under the class.

24

Multiple Inheritance & MRO

Python allows a class to inherit from several parents; the Method Resolution Order (MRO) defines the exact lookup order Python follows to find an attribute.

```
class A:
    def greet(self): return "A"
class B(A):
    def greet(self): return "B"
class C(A):
    def greet(self): return "C"
class D(B, C):
    pass

D().greet()          # "B" — follows MRO
D.__mro__             # (D, B, C, A, object)
```

- ✓ Python uses the C3 linearization algorithm to compute a consistent MRO.
- ✓ Check `ClassName.__mro__` any time inheritance behavior seems surprising.

25

Magic / Dunder Methods

Dunder (“double underscore”) methods let your classes hook into Python's built-in syntax — operators, `len()`, iteration, and more.

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)
    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

Vector(1, 2) + Vector(3, 4)    # Vector(4, 6)
```

- ✓ `__len__`, `__getitem__`, `__contains__`, `__eq__` are among the most commonly implemented.
- ✓ Implementing the right dunder methods makes a custom class behave like a built-in type.

26

__str__ vs __repr__

`__str__` returns a readable, user-facing string (used by `print()` and `str()`);
`__repr__` returns an unambiguous, developer-facing string (used by the console and `repr()`).

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __str__(self):
        return f"({self.x}, {self.y})"
    def __repr__(self):
        return f"Point(x={self.x}, y={self.y})"
```

- ✓ If `__str__` is missing, Python falls back to `__repr__` automatically.
- ✓ Aim to make `__repr__` valid Python code that could recreate the object.

27

Property Decorator (@property)

@property turns a method into a read-only attribute, letting you add validation or computed values without changing how callers access it.

```
class Circle:
    def __init__(self, radius):
        self._radius = radius
    @property
    def area(self):
        return 3.14159 * self._radius ** 2
    @property
    def radius(self):
        return self._radius
    @radius.setter
    def radius(self, value):
        if value < 0:
            raise ValueError("radius can't be negative")
        self._radius = value
```

- ✓ Callers use `circle.area`, not `circle.area()` — it looks like a plain attribute.
- ✓ Pair with a matching `@x.setter` to control how a value can be reassigned.

28

Abstract Classes (ABC)

An abstract base class defines a required interface — methods subclasses must implement — but can never be instantiated itself.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side ** 2

# Shape()    # TypeError: can't instantiate abstract class
```

- ✓ Forces every subclass to implement the required methods, catching mistakes at class-definition time.
- ✓ Commonly used to design plugin systems and enforce a consistent API.

29

Duck Typing

“If it walks like a duck and quacks like a duck, it's a duck” — Python cares about what an object can do, not its declared type.

```
class Duck:
    def sound(self): return "Quack"
class Robot:
    def sound(self): return "Beep"

def make_sound(thing):
    print(thing.sound())    # works for any object with .sound()

make_sound(Duck())
make_sound(Robot())
```

- ✓ No inheritance or interface declaration is required — just the matching method or attribute.
- ✓ This is why `isinstance()` checks are often discouraged in idiomatic Python; try the operation instead.

30

Encapsulation & Name Mangling

A single leading underscore (`_var`) signals “internal use only” by convention; a double leading underscore (`__var`) triggers name mangling to avoid subclass name clashes.

```
class Account:
    def __init__(self, balance):
        self._balance = balance      # convention: protected
        self.__pin = "1234"         # name-mangled

acct = Account(100)
acct._balance      # accessible, but "please don't"
acct._Account__pin # actual mangled name, still reachable
```

- ✓ Python has no true “private” attributes — encapsulation here is convention, not enforcement.
- ✓ Name mangling rewrites `__pin` to `_Account__pin` to reduce accidental overrides in subclasses.

31

try / except / else / finally

try runs risky code, except handles specific errors, else runs only if no exception occurred, and finally always runs — for cleanup — regardless of what happened.

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Can't divide by zero")
else:
    print(f"Success: {result}")
finally:
    print("Cleanup runs no matter what")
```

- ✓ Catch specific exceptions (ZeroDivisionError) rather than a bare except:, which hides bugs.
- ✓ else keeps success-path code out of the try block, so it isn't accidentally caught too.

32

Custom Exceptions

You can define your own exception types by subclassing `Exception`, giving callers a precise, catchable signal about what went wrong in your domain.

```
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        super().__init__(f"Balance {balance} too low for {amount}")
        self.balance = balance

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(balance, amount)
```

- ✓ Build a small exception hierarchy (e.g. `AppError` -> `ValidationError`) so callers can catch broadly or narrowly.
- ✓ Always call `super().__init__()` so `str(exception)` still works as expected.

33

Context Managers (with statement)

A context manager guarantees setup and teardown code runs — even on error — using `__enter__` and `__exit__`, or the simpler `@contextmanager` decorator.

```
class ManagedFile:
    def __enter__(self):
        self.f = open("data.txt", "w")
        return self.f
    def __exit__(self, exc_type, exc_val, exc_tb):
        self.f.close()

with ManagedFile() as f:
    f.write("hello")
# file is guaranteed closed here, even if write() raised
```

- ✓ The built-in `with open(...) as f` pattern is the most common example.
- ✓ `from contextlib import contextmanager` lets you write one with a single generator function instead of a class.

34

Global vs Local vs Nonlocal (LEGB)

Python resolves a name using the LEGB order — Local, Enclosing, Global, Built-in — checking each scope in turn until it finds a match.

```
x = "global"
def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)      # "local"
    inner()
    print(x)          # "enclosing"
print(x)              # "global"
```

- ✓ global lets a function reassign a module-level variable; nonlocal does the same for an enclosing function's variable.
- ✓ Reading an outer variable needs no keyword — only reassigning it does.

35

Modules vs Packages

A module is a single .py file; a package is a directory of modules containing an `__init__.py` file, letting you organize code into a namespace hierarchy.

```
# project structure:  
# mypackage/  
#     __init__.py  
#     utils.py  
#     models.py  
  
from mypackage import utils  
from mypackage.models import User
```

- ✓ `__init__.py` can be empty — its presence is what makes Python treat the folder as a package.
- ✓ Use relative imports (`from . import utils`) for code that lives inside the same package.

36

Virtual Environments

A virtual environment is an isolated Python installation with its own site-packages, so each project can have its own dependency versions without conflicts.

```
python -m venv .venv          # create
source .venv/bin/activate     # activate (macOS/Linux)
.venv\Scripts\activate        # activate (Windows)
pip install requests          # installs only into .venv
deactivate
```

- ✓ Never install project dependencies globally — it causes silent version conflicts across projects.
- ✓ requirements.txt (or pyproject.toml) records exactly what a project needs to reproduce the environment.

37

PEP 8 & Code Style

PEP 8 is Python's official style guide — consistent naming, spacing, and layout that make code easier for any Python developer to read.

```
# snake_case for functions/variables, PascalCase for classes
def calculate_total(price_list):
    return sum(price_list)

class ShoppingCart:
    MAX_ITEMS = 50    # constants in UPPER_CASE
```

- ✓ 4 spaces per indentation level; keep lines to roughly 79-99 characters.
- ✓ Tools like black and flake8 auto-format and lint code to PEP 8 automatically.

38

Type Hinting

Type hints annotate expected argument and return types, improving readability and letting tools like mypy catch bugs before runtime.

```
def add(a: int, b: int) -> int:
    return a + b

from typing import Optional, List
def find_user(user_id: int) -> Optional[str]:
    ...
names: List[str] = ["Alice", "Bob"]
```

- ✓ Type hints are not enforced at runtime — Python stays dynamically typed regardless.
- ✓ Static checkers (mypy, pyright) and IDEs use hints to flag mismatches before you run the code.

39

Multithreading vs Multiprocessing

Threading runs multiple threads within one process sharing memory (limited by the GIL for CPU work); multiprocessing runs separate processes, each with its own memory, for true parallelism.

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

def fetch_url(url): ...           # I/O-bound -> use threads
def crunch_numbers(data): ...     # CPU-bound -> use processes

with ThreadPoolExecutor() as ex:
    ex.map(fetch_url, urls)
with ProcessPoolExecutor() as ex:
    ex.map(crunch_numbers, datasets)
```

- ✓ Threads are lighter-weight but limited by the GIL for CPU-bound tasks.
- ✓ Processes bypass the GIL entirely but cost more memory and slower inter-process communication.

40

Enumerate, zip & Unpacking

`enumerate()` pairs items with their index while looping; `zip()` combines multiple iterables element-by-element; unpacking assigns several values from a sequence in one line.

```
for i, name in enumerate(["a", "b", "c"]):  
    print(i, name)           # 0 a / 1 b / 2 c  
  
names, ages = ["Al", "Bo"], [30, 25]  
for name, age in zip(names, ages):  
    print(name, age)  
  
first, *rest = [1, 2, 3, 4]   # first=1, rest=[2, 3, 4]
```

- ✓ `enumerate(seq, start=1)` lets you begin counting from any number.
- ✓ `zip()` stops at the shortest iterable — use `itertools.zip_longest()` to pad instead.

Decorators, Step by Step

```
import functools

def retry(times):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    print(f"Attempt {attempt+1} failed: {e}")
            raise RuntimeError("All retries failed")
        return wrapper
    return decorator

@retry(times=3)
def flaky_call():
    ...
```

- ✓ A decorator factory (`retry(times)`) is a function that returns a decorator — that's why there are three nested functions here.
- ✓ `functools.wraps` keeps `flaky_call.__name__` and `__doc__` intact instead of showing "wrapper".
- ✓ Stacking decorators (`@a @b def f`) applies them bottom-up: `f = a(b(f))`.

Generators, Step by Step

```
def fibonacci():
    a, b = 0, 1
    while True:                # infinite, but memory-safe
        yield a
        a, b = b, a + b

fib = fibonacci()
first_five = [next(fib) for _ in range(5)]
# [0, 1, 1, 2, 3]

evens = (n for n in fibonacci() if n % 2 == 0)
next(evens)    # 0
next(evens)    # 2
```

- ✓ Because `fibonacci()` never returns, only yields, it can safely represent an infinite sequence.
- ✓ Wrapping it in a generator expression (`evens`) chains lazy transformations without ever materializing the full sequence.
- ✓ Generators keep only one value in memory at a time — the opposite of building a giant list upfront.

List vs Tuple vs Set vs Dict

Type	Ordered	Mutable	Duplicates	Typical use
list	Yes	Yes	Allowed	A growing sequence of items
tuple	Yes	No	Allowed	Fixed records, dict keys
set	No	Yes	Not allowed	Membership tests, de-duplication
dict	Yes*	Yes	Unique keys	Key → value lookups

*Since Python 3.7, dicts preserve insertion order as a language guarantee — it's no longer just a CPython implementation detail.

Common Gotchas To Remember

Mutable default arguments

`def f(items=[]):` reuses the same list across every call. Use `items=None` and create the list inside the function instead.

Late binding in closures

Lambdas inside a loop all capture the same variable, not its value at creation time — pass it as a default argument to freeze it.

Catching too broadly

`except Exception:` (or a bare `except:`) silently swallows bugs. Catch the specific exception you expect.

Modifying a list while iterating it

Removing items during a for loop skips elements. Iterate over a copy (`for x in list(items)`) instead.

is for value comparison

`is` checks identity, not equality. Two equal-looking objects can be different objects — always use `==` for values.

All 40 Concepts, At a Glance

01 Mutable vs Immutable Objects

03 Shallow Copy vs Deep Copy

05 Python's Garbage Collection

07 Tuple vs List

09 String Immutability & Interning

11 *args and **kwargs

13 Closures

15 Generators and yield

17 List Comprehension vs Generator Expression

19 Recursion

21 The self Keyword

23 Class vs Static vs Instance Methods

25 Magic / Dunder Methods

27 Property Decorator (@property)

29 Duck Typing

31 try / except / else / finally

33 Context Managers (with statement)

35 Modules vs Packages

37 PEP 8 & Code Style

39 Multithreading vs Multiprocessing

02 is vs == (Identity vs Equality)

04 Pass by Object Reference

06 The GIL (Global Interpreter Lock)

08 Dictionary vs Set

10 Slicing

12 Lambda Functions

14 Decorators

16 Iterators vs Iterables

18 map(), filter(), reduce()

20 The Walrus Operator (:=)

22 __init__ vs __new__

24 Multiple Inheritance & MRO

26 __str__ vs __repr__

28 Abstract Classes (ABC)

30 Encapsulation & Name Mangling

32 Custom Exceptions

34 Global vs Local vs Nonlocal (LEGB)

36 Virtual Environments

38 Type Hinting

40 Enumerate, zip & Unpacking

NOW YOU KNOW

Python's Interview-Ready Toolkit

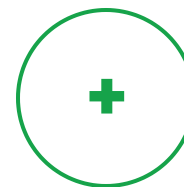
40 concepts, real examples, no fluff. Master these and you will be ready for the questions that come up again and again in Python interviews.



Like this post



Repost it



Follow for more